華中科技大學

2026

嵌入式系统　　课程实验报告

题　　目：嵌入式系统实验报告

专　　业：计算机科学与技术

班　　级：_____

学　　号：_____

姓　　名：_____

电　　话：_____

邮　　件：_____

目 录

1 实验一 系统烧录 1
1.1 实验要求 1
1.2 实验过程 1
1.3 实验结果 2
2 实验二 图形界面基础 3
2.1 实验要求 3
2.2 实验过程 3
2.3 实验结果 4
3 实验三 图片文字显示 5
3.1 实验要求 5
3.2 实验过程 5
3.3 实验结果 8
4 实验四 多点触摸开发 9
4.1 实验要求 9
4.2 实验过程 9
4.3 实验结果 10
5 实验五 蓝牙通讯 11
5.1 实验要求 11
5.2 实验过程 11
5.3 实验结果 12
6 实验六 综合实验 13
6.1 实验要求 13
6.2 实验过程 13
6.3 实验结果 16
7 实验总结与建议 17
7.1 实验总结 17
7.2 实验建议 17
7.3 实验复现与连接检查 19
7.4 源码管理与性能记录 20

1 实验一 系统烧录

1.1 实验要求

完成 Orange Pi 4 LTS 的 Linux 系统烧录和启动，建立电脑与开发板之间的连接，熟悉课程工程的编译、部署和运行方法，并在开发板上运行第一个应用程序。实验依据课程 PPT 第 19—43 页。

1.2 实验过程

（1）烧录系统与连接硬件

我在 Windows 电脑上解压课程提供的 Orangepi4-lts_3.0.6_ubuntu_desktop_otg.img 镜像，使用 balenaEtcher 选择镜像和约 64 GB 的存储卡，完成写入及校验。出现“Flash Complete!”后取出存储卡，在开发板断电状态下将其插入卡槽，再连接开发板电源、显示屏电源和 HDMI，等待系统启动。

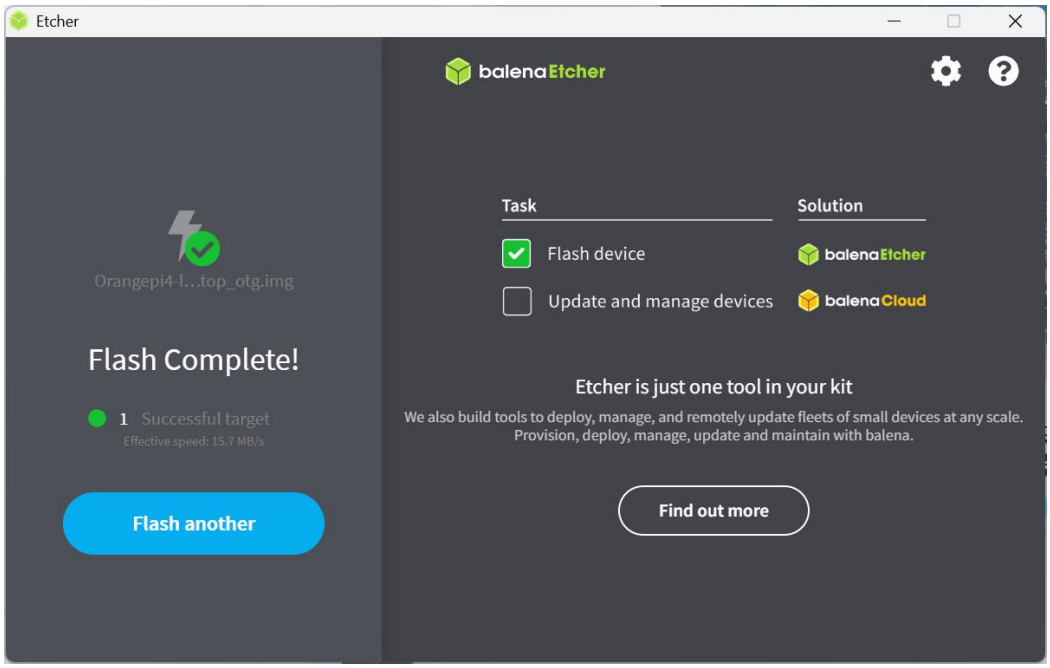


图 1-1 balenaEtcher 烧录并校验完成。

（2）建立远程连接

实验中，我主要采用网线连接电脑与开发板，将电脑的 WLAN 网络共享给以太网接口，再通过 Windows PowerShell 中的 SSH 客户端登录。电脑以太网地址设为 192.168.137.1，开发板 eth0 地址设为 192.168.137.115。后续调试时也使用过手机热点，当时开发板的地址为 192.168.43.160。

我在 PowerShell 中执行 ssh root@192.168.137.115，首次连接时确认主

机信息，再输入密码。开始时我误以为键盘没有响应，后来发现 SSH 输入密码时本来就不显示字符，输入完成后按回车即可。登录成功后，终端出现 root@orangepi4-lts 提示符，我便可以直接在电脑上编译和运行开发板程序。

（3）编译并运行程序

我将课程工程复制到开发板，把 common/rules.mk 中的编译器设为 gcc，使用板端 GCC 进行编译。进入 lab1 目录执行 make，再进入 out 目录运行 ./lab1，最后执行 echo $? 查看程序返回值。

1.3 实验结果

系统烧录后，开发板成功启动 Orange Pi 3.0.6 Focal，内核版本为 Linux 5.10.43，体系结构为 aarch64。通过 SSH 登录并运行 lab1，终端输出“Hello embedded linux!”，返回值为 0。至此完成了系统烧录、远程连接和首个程序运行，并通过助教验收。

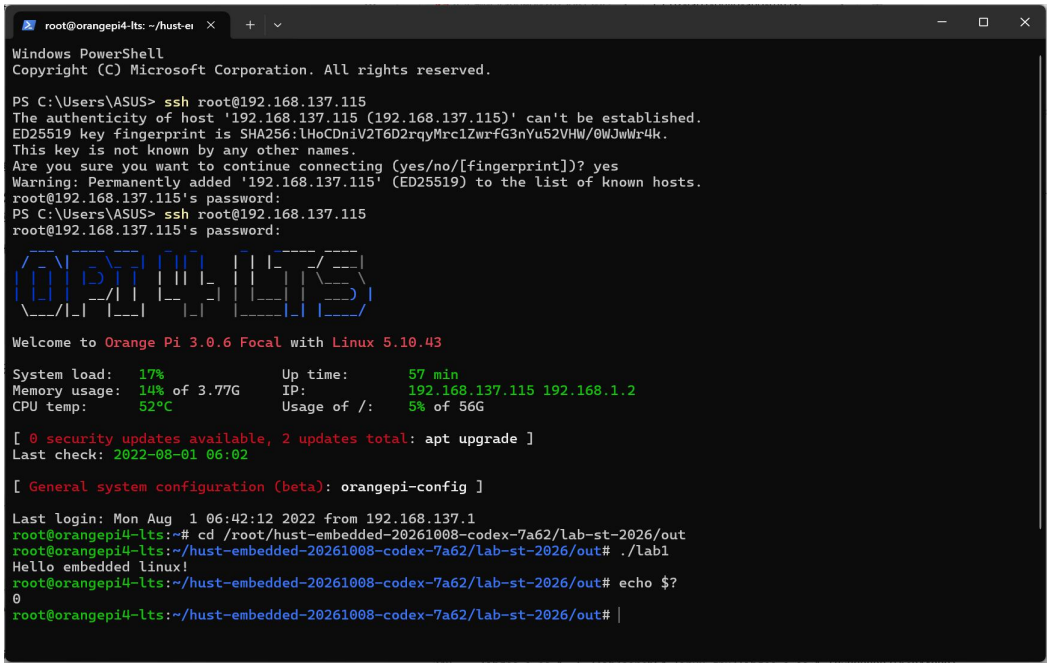


图 1-2 通过 PowerShell 登录开发板并运行 lab1，程序正常退出。

2 实验二 图形界面基础

2.1 实验要求

理解 Linux framebuffer 的显示原理与内存映射方式，补全矩形和直线绘制函数，在屏幕上显示点、线和矩形，并理解软件双缓冲及更新区域的作用。实验依据课程 PPT 第 44—52 页。

2.2 实验过程

（1）初始化 framebuffer

我先打开 /dev/fb0，通过 ioctl 获取分辨率、像素格式和行跨度，再使用 mmap 将显存映射到进程地址空间。开发板的可见分辨率为 1024×600，每像素 32 位，虚拟分辨率为 1280×720，line_length 为 5120 字节。计算像素地址时，我使用实际行跨度定位每一行，再加上横坐标对应的字节偏移，解决了可见宽度与显存布局不一致的问题。

（2）补全矩形绘制

我在 fb_draw_rect 中根据起点、宽度和高度确定矩形范围，将超出屏幕的部分裁去，再逐行填充颜色。矩形每一行对应一段连续内存，因此只需计算一次行首地址，再顺序写入各像素。与反复调用画点函数相比，按行填充减少了重复的地址计算与函数调用，绘制宽度为 w、高度为 h 的矩形仍需完成 O(w×h) 次像素写入。

（3）补全直线绘制

我在 fb_draw_line 中采用整数误差累加的方法绘制直线。程序选择坐标变化较大的方向作为主轴，每沿主轴前进一步，就根据累计误差判断另一方向是否移动，从而生成连续的像素轨迹。整个过程使用整数加减与比较，不必逐点计算浮点斜率，能够统一处理水平线、竖直线和不同斜率的线段；两个端点重合时绘制一个点。

（4）更新屏幕并运行测试

我将绘制与显示更新分开：先把图形写入内存缓冲并记录更新区域，再由 fb_update 按显存的实际行跨度复制到 framebuffer。运行程序前，我临时停止开发板桌面，使实验程序直接控制屏幕。

2.3 实验结果

程序在开发板上编译、运行成功，屏幕能够显示点、矩形和不同方向的直线。画点、矩形和直线三个测试阶段分别耗时 1 ms、2 ms 和 2 ms，程序返回值为 0，实验通过助教验收。

测试项目	阶段耗时/ms
画点	1
矩形	2
直线	2

表 2-1 实验二各绘图阶段的运行结果

3 实验三 图片文字显示

3.1 实验要求

实现不透明 JPEG 图像、带透明度的 PNG 图像以及矢量字体的显示，正确处理颜色格式、透明度混合和 UTF-8 字符，运行综合图形测试并分析性能。实验依据课程 PPT 第 53—64 页。

3.2 实验过程

（1）图像和文字绘制

我用 fb_image 统一保存图像格式、宽高、行跨度和像素数据，并分别实现 JPEG 和 PNG 的绘制。JPEG 按不透明图像直接写入，PNG 则根据 alpha 值混合前景色与背景色。完全透明的像素直接跳过，完全不透明的像素直接写入，其余像素采用整数运算完成颜色混合。

我使用 FreeType 从 font.ttc 中读取字形，按 UTF-8 编码逐个解析字符，再根据 left、top 和 advance_x 确定字形位置与下一个字符的起点。字模中的灰度值作为透明度，与指定颜色组合后绘制到背景上。调试小字号文字时，我发现单色 MONO 字模与 8 位灰度字模的存储方式不同，补充格式转换后解决了显示问题。



图 3-1 JPEG、透明 PNG 与中文文字显示结果

（2）优化重复工作

我首先为相同字符和字号建立字形缓存，减少重复解析。随后为不同图像格

式设置专用绘制路径，分别处理透明、不透明和半透明像素；对重复绘制的透明图像建立目标行索引，减少无效遍历。像素计算进一步结合 ARM NEON 和按行并行处理，以缩短图片与文字的绘制时间。

对于多张不透明图像连续覆盖同一区域的情况，我将绘制请求合并为批量任务，分析每个区域最终由哪次绘制保留，只写入最终可见的部分。这样可以省去被后续图像完全覆盖的中间写入。该路径在批次结束后统一输出画面，适合本次重复图像绘制测试。

我对矩形的不同画法进行上板对比，最终保留耗时较低的直接绘制方案。同时比较优化前后的像素结果，并检查图片、透明效果和中文显示，确认优化后的画面正常。这个过程让我认识到，减少绘制次数和加快一次绘制是两类优化，需要结合实际数据选择。

从工作量看，我把优化分为减少单次计算和减少重复绘制两部分。若重复绘制 N 次,每次图像包含 P 个像素,逐次完整遍历需要处理约 N×P 个像素。NEON 和按行并行主要加快这部分计算，仍然需要处理相同数量的像素；字形缓存则把重复的字符解析与栅格化结果保存起来,后续直接复用。这样既减少了函数调用，也把更多时间留给实际的像素写入。

批量覆盖消除进一步改变了需要执行的工作。以多次在同一位置绘制同样大小的不透明图像为例，如果两次绘制之间没有刷新屏幕，最终画面只保留最后一次图像。我的实现先整理绘制请求，确定最后可见的区域，再执行这些区域的写入。这个例子中的像素写入量可以从 N×P 降到接近 P，另外仍有请求整理与区域分析的开销。实际收益与图像位置、覆盖关系和可见区域数量有关，这也解释了为什么这种方法在重复覆盖较多时效果明显。

透明 PNG 的处理方式有所不同：半透明像素会读取背景并参与混合，连续绘制的结果与先后顺序有关。因此，我仍保留透明混合的顺序，主要从跳过透明像素、复用行索引和加快颜色运算入手。文字部分则复用字形与布局计算。对照最快一轮的数据，PNG 为 6 ms，矩形和直线各为 4 ms，已经占到总成绩的大部分；后续继续优化时，我会优先分析这些阶段。画点显示 0 ms 是整数毫秒计时的结果，说明该阶段很短。

（3）性能测试

课程测试程序分别记录画点、矩形、直线、JPEG、PNG、字模和文本七个阶段的整数毫秒耗时，总成绩为这七项之和，图像解码和屏幕刷新在计时范围之外。测试时记录所用版本、编译选项及运行配置，并将批量绘制版本与早期版本分别列出。

我让每组测试连续运行三轮，记录各分项和总耗时，再计算中位数，用于观察优化效果及运行波动。比较版本时，除总成绩外，我还逐项检查七个阶段的变化，以判断新的优化究竟节省了哪部分工作。



图 3-2 批量绘制优化后总耗时为 17 ms 的运行截图

3.3 实验结果

开发板正常显示羊驼 JPEG、绿色透明 PNG、红色“嵌”字和紫色《静夜思》。早期加入字形缓存并开启 -O3 优化后，三轮测试耗时为 160、155、173 ms，中位数为 160 ms。

采用批量绘制优化并保留直接矩形绘制后，三轮耗时为 22、21、17 ms，中位数为 21 ms。其中最快一轮的分项为：画点 0 ms、矩形 4 ms、直线 4 ms、JPEG 1 ms、PNG 6 ms、字模 1 ms、文本 1 ms，合计 17 ms。

后续再次运行同一部署版本，三轮结果为 58、19、20 ms，中位数为 20 ms。两组结果表明，优化后的典型耗时降至约 20 ms，首次运行仍存在一定波动。

版本	三轮耗时/ms	中位数/ms	最小值/ms	测试配置
早期基线	228 / 236 / 263	236	228	早期原接口
字模缓存	170 / 172 / 175	172	170	同批字模缓存
缓存及-O3	160 / 155 / 173	160	155	缓存与 -O3
同轮基线	47 / 46 / 47	47	46	同配置原接口
v9-sprint	27 / 27 / 25	27	25	批量绘制
v10-sprint	28 / 25 / 28	28	25	批量绘制
直接矩形组合	22 / 21 / 17	21	17	直接矩形＋批量
已部署最优组合复测	58 / 19 / 20	20	19	同部署版本复测

表 3-1 不同版本的三轮测试耗时

4 实验四 多点触摸开发

4.1 实验要求

读取 Linux Input Event 和多点触摸协议中的触点信息，在 LCD 上绘制触摸轨迹。本实验选择课程 PPT 第 72 页的连续轨迹方案：不同手指使用不同颜色，线条连续且具有明显宽度，并提供清屏按钮。相关原理见 PPT 第 65—72 页。

4.2 实验过程

（1）识别触摸设备

我通过 /proc/bus/input/devices 和 /dev/input/eventN 查找触摸设备，识别到的屏幕型号为 wch.cn USB2IIC_CTP_CONTROL。屏幕通过 HDMI 接收图像，通过 USB 数据线向开发板传送触摸事件。调试时，我根据设备信息确定对应的 event 节点。

（2）解析多点事件

我按 Protocol B 为各 slot 保存跟踪编号、坐标和按压状态，并在同步事件到来时生成按下、移动和抬起等应用事件。触摸原始坐标范围为 0—4096，我将其映射到 1024×600 的显示坐标。驱动报告 6 个 slot，课程应用使用其中 0—4 共五个触点，各触点分别维护自己的状态。

（3）绘制连续轨迹与清屏

我为五个触点分别分配红、蓝、绿、紫、橙五种颜色，并记录各触点的上一次坐标。在相邻采样点之间插值绘制 9 像素笔刷，使手指快速移动时也能形成连续粗线。手指抬起时结束当前笔画，再次按下时开始新笔画。



图 4-1 五指画板的五种颜色连续轨迹。

画板右上角设置清屏按钮，点击后清空画布，下一次触摸开始新的笔画。顶部色块展示五个触点对应的颜色。

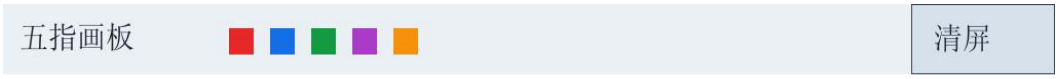


图 4-2 空白画板及清屏按钮界面（程序初始状态截图）。

4.3 实验结果

我在屏幕上依次测试单指快速画线、五指同时拖动和清屏。单指轨迹连续，五指能够同时画出五条不同颜色的粗线，清屏后画布恢复为空白。事件统计中的最大同时触点数为 5，与屏幕上的操作结果一致。

5 实验五 蓝牙通讯

5.1 实验要求

配置 Linux 蓝牙设备，理解配对、服务发现、RFCOMM 连接与应用数据收发的关系，通过蓝牙串口实现双向通信。按照课程 PPT 第 85 页给出的方案，本实验使用安卓手机与开发板进行通信。实验整体依据 PPT 第 73—87 页。

5.2 实验过程

（1）准备蓝牙服务与手机应用

我在开发板上配置 BlueZ，开启经典蓝牙的可发现和可配对状态，并配置串口服务。手机使用带有“SPP”标识的蓝牙串口应用。我先在手机系统设置中与开发板配对，再进入串口应用建立数据连接。

（2）建立 RFCOMM 数据通道

我在开发板运行 rfcomm -r watch 0 1 监听连接。手机串口应用连接后生成 /dev/rfcomm0，我让程序打开该设备文件进行读写，界面状态由等待连接切换为 Connected。

（3）收发文字与处理断线

我在接收端拼接分段到达的 UTF-8 数据，处理换行后将文字显示到屏幕上，并为 send 按钮添加发送 hello 的回调。收发过程结合事件回调与发送队列进行，保持界面能够继续响应触摸。连接断开后，程序返回等待状态，重新连接后恢复收发。

（4）通信测试

我先在电脑端通过伪终端测试文本收发，再使用手机与开发板进行蓝牙测试。手机发送“你好，orange pi！”，开发板显示对应文字；点击开发板 send，手机收到 hello。随后在手机应用中断开连接，再重新连接并重复双向收发。

5.3 实验结果

连接成功后，开发板显示 Connected，手机发送的中文能够正常显示，手机也能收到开发板发送的 hello。断开并重新连接后，两个方向的文字收发仍正常，完成了蓝牙串口通信及重连测试。



图 5-1 开发板蓝牙串口已连接并收到中文消息。

6 实验六 综合实验

6.1 实验要求

课程 PPT 第 88 页要求综合运用前面实验的技术，设计多开发板通信程序或功能较复杂的单机程序。我选择单机游戏方向，在 Orange Pi 4 LTS 上实现“橙派游戏工坊 v2”，整合五子棋、2048 和三关打砖块，提供触摸操作、中文界面、成绩记录与存档功能。

6.2 实验过程

（1）整体设计与模块划分

我将游戏工坊划分为硬件接入、交互状态、界面绘制和游戏规则等模块。arcade/main.c 负责硬件初始化、触摸回调和定时更新；app.c 管理菜单与交互状态；render.c 绘制界面；engine.c 实现 2048 规则与提示；breaker.c 实现打砖块的关卡和碰撞；profile.c 保存成绩记录。五子棋使用独立的 lab6_gomoku 工程，我分别实现棋规、AI、交互、绘制和存档，再通过统一菜单组织三种游戏。

我编写配套脚本负责游戏之间的切换。从菜单选择五子棋后，脚本启动五子棋程序；退出对局后，再返回游戏工坊菜单。这样的组织方式便于复用原有五子棋模块，也使各游戏的逻辑相对独立。

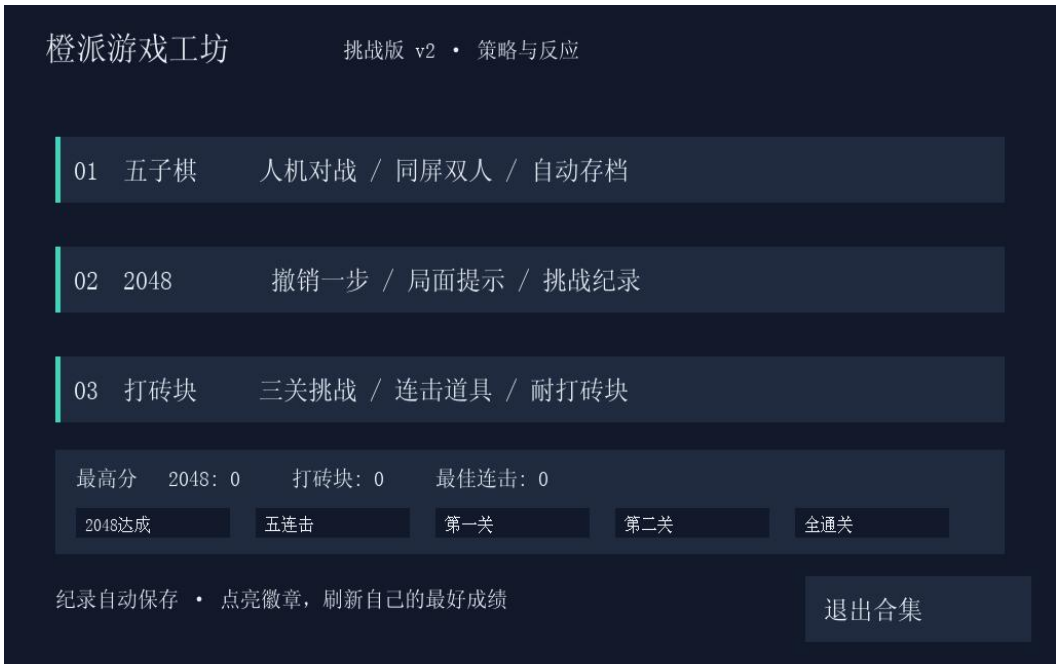


图 6-1 游戏工坊 v2 统一菜单。

（2）统一显示与触摸交互

我复用前面实验的显示与触摸接口，让游戏统一使用 1024×600、32 位 RGB framebuffer，并按实际 line_length 更新显存。触摸坐标经映射后传递给交互层，由当前主触点控制按钮和滑动。菜单、2048 等页面在状态变化时重绘，打砖块通过定时事件持续更新，中文显示则复用实验三的字体绘制功能。

（3）五子棋模块

我将五子棋棋盘设为 15×15，采用自由规则，横、竖和两条斜线方向连续五子或以上获胜。程序支持人机对战、同屏双人、悔棋、重新开始、最后落子提示及保存退出。每次触摸按下只落一子，已经占用的位置不能落子。

我为五子棋 AI 设计了先处理直接胜负、再搜索候选点的决策流程。程序先检查自己是否能够一步获胜，以及对手是否存在需要立即阻挡的威胁，再对已有棋子附近的候选点评分。评分考虑连子长度、两端开放情况和五格窗口中的断续棋形，并对得分较高的候选点进一步搜索对手的应答。我把搜索拆成定时任务分步完成，使电脑思考时界面仍能响应操作。

我用对局模式和完整落子序列保存五子棋进度，重新打开时按顺序回放落子，恢复棋盘与当前回合。这样可以在退出程序后继续之前的对局。

（4）2048 模块

我用 4×4 数组表示 2048 棋盘，将四个滑动方向转换为逐行处理次序，再按“压紧—相邻同值合并—补零”完成移动。每个格子在一次移动中只参与一次合并，例如 [2,2,2,2] 向左移动后得到 [4,4,0,0]，两次合并共增加 8 分。仅在棋盘发生有效变化后生成新格子，生成 2 和 4 的概率分别为 90% 和 10%。

我为 2048 添加撤销和提示功能。撤销时恢复上一步的棋盘、分数和随机数状态；提示时比较四个移动方向，结合随机落点及下一步选择，按照空格数量、相邻数值差和大数靠角等因素评分，给出建议方向。有效移动后的格子边框短暂高亮，便于观察棋盘变化。



图 6-2 2048 的撤销与提示界面

（5）三关打砖块模块

我设计了三个逐步变化的关卡：第一关使用普通砖阵，第二关采用交错阵型，第三关引入需击打两次的耐打砖。触摸拖动控制挡板，松手发球；程序处理球与边界、挡板和砖块的碰撞，维护得分、生命、当前关卡和过关状态。过关后通过“进入下一关”继续，保留当前分数与生命。

连续击破砖块提高连击得分，碰到挡板或漏球时重置连击。每累计击破五块砖且没有活动道具时，依次产生“宽”“缓”“盾”三种道具；挡板接住后分别使挡板加宽 10 秒、球速降至 68%并持续 8 秒，或抵挡一次漏球。道具需要实际接住才生效，漏球与切关按规则清理状态。

我将打砖块的物理计算设为 1/240 秒固定时间步，画面由 20 毫秒定时器驱动。每一步检测球与边界、挡板和砖块的碰撞，并更新球拖尾和粒子效果。将物理计算与画面更新分开，有助于保持运动与碰撞效果稳定。

（6）成绩记录与存档

我为游戏工坊加入 2048 最高分、打砖块最高分、最佳连击和五枚挑战徽章的记录功能，重新启动后可继续查看。2048 与打砖块记录历史成绩，五子棋则单独保存完整对局，便于续玩。

（7）上板调试

我先在电脑端测试游戏规则和界面逻辑，再把工程复制到开发板进行 ARM 原生编译。首次独立打包时遇到缺少 common/task.c 的问题，在构建脚本中补充课程任务调度文件后，游戏工坊与五子棋均编译成功。随后检查 framebuffer 和触摸设备，启动程序进行试运行。

6.3 实验结果

游戏工坊 v2 在开发板上编译成功，自动测试通过了 8000 次 2048 移动检查、200 个提示局面检查、100000 帧模拟物理测试以及文件读写测试。

调试时曾出现画面正常但触摸无响应的问题，重新接好屏幕与开发板之间的 USB 数据线后，系统识别到 wch.cn USB2IIC_CTP_CONTROL，对应节点为 /dev/input/event2。游戏工坊启动后能够进入打砖块，屏幕正常显示砖阵、挡板、球和中文菜单。图 6-3 记录了第 1 关、350 分、剩余 2 条生命时的运行画面。

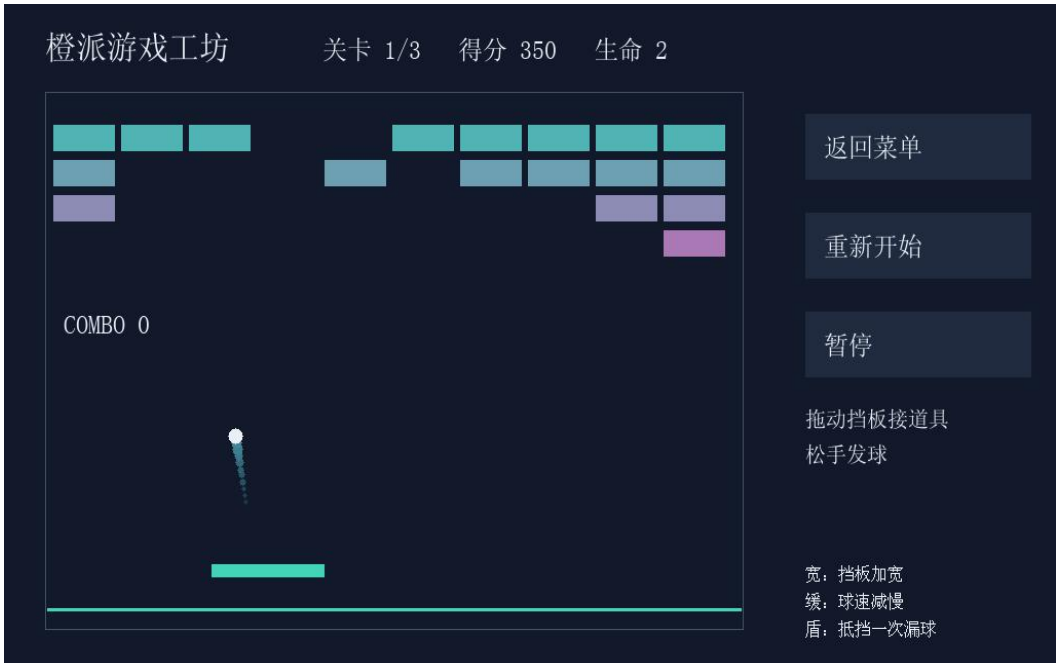


图 6-3 游戏工坊 v2 打砖块运行画面

五子棋试运行中，点击棋盘后黑棋落在对应交叉点，电脑随后落下白棋，悔棋操作也正常。实际对弈时，电脑能够识别并阻挡连子威胁，达到了预期的人机对战效果。

7 实验总结与建议

7.1 实验总结

本次实验从系统烧录和远程连接开始，逐步完成基础绘图、图片文字显示、多点触摸和蓝牙通信，最后组合成游戏工坊。我对嵌入式应用从环境搭建、接口调用到功能整合的过程有了更完整的认识，也体会到将各个实验模块组合起来时，需要同时考虑程序结构和实际操作体验。

连接与排障是实验中花费时间较多的部分。我遇到过网络地址设置错误、SSH 无法登录，以及屏幕能显示但无法触摸等问题。逐步检查网卡状态、IP 地址、设备节点和接线后，问题得到解决。尤其是 HDMI 与触摸 USB 分别负责显示和输入，这让我更直观地理解了硬件接口与软件功能之间的对应关系。

实验三让我体会到，优化前应先找出重复工作较多的环节。字形缓存减少了解析次数，图像专用路径与并行处理加快了像素运算，批量覆盖消除进一步减少了无效写入。经过多轮测试，总成绩从早期约 160 ms 降至约 20 ms。记录分项耗时和三轮结果，帮助我判断哪些修改有效，也让我注意到首次运行带来的波动。

综合实验中，我把前面完成的显示、文字和触摸功能用于游戏开发，并进一步实现规则判断、人机搜索、定时动画与存档。五子棋侧重策略搜索，2048 侧重状态转换，打砖块侧重碰撞与连续画面更新。将三种游戏组织到同一菜单中，使我进一步熟悉了模块划分、事件处理和程序状态切换。

7.2 实验建议

结合自己的接线经历，我建议在实验开始前提供更直观的硬件接线说明，将开发板供电、显示屏供电、HDMI 视频线、触摸 USB 数据线和网络连接分别标注，并配上接口照片。本次实验中，显示正常但触摸无响应的问题，最终通过重新连接屏幕与开发板之间的 USB 数据线得到解决。如果指导材料能够明确每根线的用途，并列出正常连接时的现象，可以减少初次使用开发板时的摸索时间。

在开发环境配置方面，建议整理一份从系统烧录到运行首个程序的快速操作单，集中说明镜像版本、编译方式、工程目录、依赖库和常用命令。对于 Windows 网络共享、SSH 登录以及手机热点连接，可以分别给出检查步骤。例如，先检查网卡是否连接，再核对电脑与开发板的 IP 地址，最后检查 SSH 服务。还可以补充密码输入不回显、开发板时钟未校准、触摸设备节点随插拔变化等常见现象，

使学生能够判断哪些属于正常行为，哪些需要进一步排查。

对于图形显示和触摸实验，建议增加一些有代表性的测试场景，例如不同斜率的直线、带透明度的图片、不同字号的中文，以及快速画线和多指同时操作。显存行跨度也值得单独举例说明：本次开发板的可见宽度为 1024 像素，但每行显存为 5120 字节，直接按屏幕宽度计算容易画错位置。把这些问题与实际画面对应起来，更有助于理解相关接口。

性能测试方面，建议统一测试素材、循环次数、编译选项和计时范围，并说明缓存、并行处理及批量绘制的比较方式。结果可以同时列出各阶段耗时和多轮测试数据，方便学生看出时间主要花在哪里。本次优化中，分项计时比单看总成绩更有帮助，可以据此确定下一步应优先优化图片、文字还是基础图形。

对于蓝牙通信实验，建议将检查流程拆分为设备发现、系统配对、SPP 应用连接、RFCOMM 设备建立和双向收发几个阶段，并为每个阶段列出预期现象与常见错误。验收时除正常收发外，还可以加入断开后重新连接的测试，观察程序是否能够恢复通信。通过这种分阶段的方式，学生更容易理解蓝牙协议与应用程序之间的关系，也能更准确地定位“已经配对但无法通信”等问题。

综合实验建议进一步明确功能完成度、交互体验和算法设计的评价要求，为游戏、工具或通信应用等不同方向提供参考。还可以安排简短的项目展示，让学生交流各自的实现思路和调试方法。报告模板、需要提交的截图及数据最好在实验开始时一并给出，便于在操作过程中及时整理材料，减少最后集中补充的工作。

7.3 实验复现与连接检查

我把两次实验课和后续试运行中用到的连接方法整理成固定流程。开始时先确认开发板和显示屏分别供电，再连接 HDMI 与触摸 USB 数据线。HDMI 负责把图像送到屏幕，触摸数据则通过 USB 返回开发板；因此，看到桌面只能说明显示链路已经工作，还需要检查触摸设备是否被系统识别。

使用网线连接笔记本与开发板时，我先在 Windows 中查看以太网是否已连接，再把无线网络共享给对应的以太网接口。开发板端用 ip -br addr 检查 eth0 的状态和地址，确认两端处于同一网段后再登录 SSH。排查时我曾把地址中的 168 写成 16，导致线路看似正常却无法连通；逐段核对地址后恢复了连接。

没有网线时，我采用手机热点连接电脑与开发板。热点分配的地址可能变化，因此每次开机后都重新查看开发板地址，再在电脑 PowerShell 中执行 ssh root@开发板地址。SSH 终端中输入密码时不会显示字符或星号，输入完成后直接回车即可。登录后看到开发板的命令提示符，才能继续在板上编译或启动实验程序。

开始显示或触摸实验前，我检查当前运行的程序，避免桌面和实验程序同时改写 framebuffer。需要停止的是开发板上的图形桌面，笔记本仍可正常使用。触摸节点通过 /proc/bus/input/devices 与 /dev/input/event* 对照查找，不把上一次的 event 编号直接当作本次编号。出现画面正常、触摸无响应时，我先检查屏幕到开发板的 USB 数据线。

实验五按设备发现、系统配对、串口应用连接和双向收发的顺序操作。手机显示已配对后，还要在 SPP 应用中连接串口服务；开发板出现 /dev/rfcomm0 后再检查应用收发。演示结束前，我会主动断开并重连一次，分别发送中文和 hello，确认重连后的通信正常。

结束实验时，我先退出前台程序，保存测速日志与运行截图，再通过终端让开发板正常关机，最后收起电源和数据线。按这套顺序准备，可以把课堂时间集中用于程序演示和问题分析，也便于下一次恢复相同的运行环境。

7.4 源码管理与性能记录

在持续优化过程中，我保留原始版本、每次阶段性修改及对应测试材料。稳定版本用于上课演示，候选版本单独编译和测试；只有结果和显示效果都检查完成后，才更新演示程序。这种组织方式使我在实验三修改绘图路径时，仍能随时回到原接口版本进行对照。

实验三的记录同时保存三轮总耗时和分项耗时。直接矩形组合的三轮结果为 22、21、17 ms，中位数为 21 ms；后续同一部署版本复测得到 58、19、20 ms，中位数为 20 ms。两组结果说明较快成绩可以再次出现，但首次运行仍会带来明显波动。对外展示时，我把完整三轮数据与最低成绩放在一起，便于看清稳定程度。

比较不同版本时，我先确认编译选项、CPU 设置、测试循环和计时位置，再分析算法变化。早期缓存版本的 160、155、173 ms，与后来的同配置基线 47、46、47 ms，不能直接视为单一代码修改造成的差异。我在记录中分别列出历史基线、同轮基线和优化版本，避免把环境变化混入算法收益。

从工作量看，逐次绘制需要处理每次请求覆盖的像素，而批量路径可以先确定最终可见区域，再集中写入显存。若多次绘制覆盖同一位置，后画内容会替代前画内容，消除这些最终不可见的写入就能减少实际像素操作。图片与文字的处理仍受像素量和字形大小影响，优化重点是减少重复解析、分支判断、数据搬运和无效覆盖。

我还把 PNG、中文文字和基础几何图形的测试与速度测试分开保存。速度下降并不自动说明功能正确，因此仍需要检查透明区域、半透明混合、字形边缘、坐标裁剪以及不同方向的直线。综合实验则增加游戏规则、撤销、存档和连续碰撞等检查，避免只依据一张正常画面判断全部功能。

最终整理时，我把代码、构建说明、连接排障、测速表和实验报告放在同一仓库，并在 README 中给出入口。公开材料保留实现方法与实测结果，课程镜像、字体和外部库仍从课程渠道取得。这样既保留了本次实验的完整过程，也便于以后按明确的依赖和操作步骤重新运行。